

Workout App — MVP Specification

Flutter, local-first, JSON-driven workouts

Working title: *TBD*

Version 0.1 | 2 October 2026

Goal	A personal-use app where a workout is created by pasting or importing JSON, then performed through a guided player with rest timers and set logging.
Platforms	Android and iOS from one Flutter codebase. Build for the phone you own first.
Backend	None. All data is stored on the device.
Done when	You have used it for every workout for two weeks without reaching for another app.

Contents

1	Product summary	3
2	Scope	3
2.1	In scope for the MVP	3
2.2	Out of scope for the MVP	3
3	User stories	3
4	Workout JSON schema (version 1)	4
4.1	Example	4
4.2	Root object	5
4.3	Workout object	5
4.4	Exercise object	5
4.5	Validation behaviour	5
5	Screens	6
6	Workout player	6
6.1	Layout	6
6.2	States	7
6.3	Rest timer and breathing cue	7

7	Reminders	7
8	Technical design	8
8.1	Suggested stack	8
8.2	Project structure	8
8.3	Design rules	8
8.4	Data model	9
8.5	Platform notes	9
9	Milestones	9
9.1	Checklist for today	9
10	Acceptance criteria	10
11	Risks	10
12	Post-MVP backlog	10

1 Product summary

Most workout apps make the user tap through many screens to build a routine. This app treats a workout as a small JSON document. The user pastes it, the app validates it, shows a preview, and saves it. Because the format is simple and documented, a workout can be written by hand, shared as a file, or generated by an AI assistant.

The MVP proves two things:

1. Importing a workout from JSON is faster and more pleasant than building one with forms.
2. The in-workout player is good enough to use for real training sessions.

2 Scope

2.1 In scope for the MVP

1. **JSON import:** paste from clipboard or pick a `.json` file; validate; preview; save.
2. **Workout library:** list, view, and delete saved workouts.
3. **Workout player:** tap an exercise to start it, mark each set done, log actual reps and weight.
4. **Rest timer:** starts automatically after a set, shows a breathing cue, and alerts the user when rest ends, including when the screen is locked.
5. **History:** list of completed sessions, and “last time” values shown in the player.
6. **Reminders:** weekly workout reminders on chosen days and time.
7. **Export:** copy a saved workout back out as JSON.

2.2 Out of scope for the MVP

Deferred item	Reason
Apple Watch / Wear OS apps	Needs native code per platform; large effort. Planned after release.
Heart rate	Depends on the watch apps.
Motivational quote notifications	Low value until reminders are proven; add as an opt-in later.
Form-based workout editor	The MVP tests the JSON-first idea. Edit by re-importing.
Accounts, cloud sync, backup	No backend in the MVP.
Supersets and circuits	Schema reserves a field for them; the player ignores it for now.
Charts and analytics	History list is enough to start.
Exercise library with images or video	Content work, not needed for personal use.

3 User stories

US1. As a user, I can paste JSON and see a readable preview of the workout before saving it.

- US2. As a user, when my JSON is wrong, I am told which field is wrong and why.
- US3. As a user, I can start a saved workout and see all its exercises in order.
- US4. As a user, I can tap an exercise to make it the active one, in any order.
- US5. As a user, I can mark a set done and adjust the reps and weight I actually achieved.
- US6. As a user, a rest timer starts automatically after each set and tells me when to go again, even if my phone is locked.
- US7. As a user, I can skip rest or add 15 seconds.
- US8. As a user, I can see what I lifted the last time I did this exercise.
- US9. As a user, I can finish a workout and find it in my history.
- US10. As a user, I get a reminder on the days and time I chose.

4 Workout JSON schema (version 1)

This format is the core of the product. Keep it small, and never change the meaning of an existing field without raising `schemaVersion`.

4.1 Example

```
{
  "schemaVersion": 1,
  "workouts": [
    {
      "name": "Push Day A",
      "description": "Chest, shoulders, triceps",
      "tags": ["push", "upper"],
      "defaultRestSeconds": 90,
      "exercises": [
        {
          "name": "Bench Press",
          "type": "reps",
          "sets": 4,
          "reps": 6,
          "repsMax": 8,
          "weight": 60,
          "unit": "kg",
          "restSeconds": 120,
          "notes": "Pause briefly on the chest"
        },
        {
          "name": "Overhead Press",
          "type": "reps",
          "sets": 3,
          "reps": 10
        },
        {
          "name": "Plank",
          "type": "timed",
          "sets": 3,
          "durationSeconds": 45,

```

```

    "restSeconds": 60
  }
]
}
]
}

```

4.2 Root object

Field	Type	Req.	Rules
schemaVersion	integer	yes	Must be 1. Higher values are rejected with “update the app”.
workouts	array	yes	One or more workout objects.

4.3 Workout object

Field	Type	Req.	Rules
name	string	yes	1–60 characters after trimming.
description	string	no	Up to 300 characters.
tags	array of string	no	Up to 10 tags.
defaultRestSeconds	integer	no	0–600. Defaults to 90.
exercises	array	yes	One or more exercise objects.

4.4 Exercise object

Field	Type	Req.	Rules
name	string	yes	1–60 characters after trimming.
type	string	no	"reps" or "timed". Defaults to "reps".
sets	integer	yes	1–20.
reps	integer	if reps	1–200. Target reps, or the lower bound of a range.
repsMax	integer	no	Upper bound of a range. Must be $\geq$ reps.
durationSeconds	integer	if timed	1–3600.
weight	number	no	≥ 0 . Suggested starting weight.
unit	string	no	"kg" or "lb". Defaults to the app setting.
restSeconds	integer	no	0–600. Overrides the workout default.
notes	string	no	Up to 300 characters.
group	string	no	Reserved for supersets. Stored but ignored in the MVP.

4.5 Validation behaviour

- Collect *all* errors in one pass and show them together, not one at a time.

- Every error names its location in plain words, for example: “*Push Day A, exercise 3 (Plank): durationSeconds is required for timed exercises.*”
- Malformed JSON reports the line and column from the parser.
- Unknown fields are ignored with a warning, so files from newer versions degrade gracefully.
- Nothing is saved until the user confirms on the preview screen.
- Importing a workout whose name already exists asks: replace, keep both, or cancel.

5 Screens

Screen	Contents and behaviour
Home / Library	List of saved workouts (name, exercise count, last performed). Import button. Tap a workout to open its detail. Empty state explains import and offers a sample workout.
Import	Large text field with “Paste from clipboard” and “Choose file” buttons. “Validate” shows either the error list or the preview.
Import preview	Read-only rendering of each workout as the player will show it. “Save” and “Back” buttons.
Workout detail	Exercise list with sets, reps, and rest. Buttons: Start, Export JSON, Delete.
Player	See Section 6.
Session summary	Duration, sets completed, total volume. “Done” returns to Home.
History	Past sessions, newest first. Tap to see logged sets.
Settings	Default unit (kg/lb), reminder days and time, rest-end sound and vibration toggles.

6 Workout player

6.1 Layout

- Scrollable list of exercise cards. Each card shows name, target, and set progress (for example 2/4).
- Tapping a card makes it the **active exercise** and expands it to show its sets.
- Each set row shows target reps and weight, the previous session’s values, and a large “Done” button.
- Reps and weight are adjusted with +/– steppers, pre-filled from the last session or the JSON.
- Controls must be large enough to use one-handed in a gym. Keep the screen awake during a session.

6.2 States

State	Behaviour
Idle	No active exercise. User picks one.
Performing	A set is in progress. For timed exercises, a count-down runs. “Done” logs the set.
Resting	Full-screen rest view with count-down ring, breathing cue, next set preview, “Skip” and “+15 s”. Ends automatically, then returns to Performing.
Exercise complete	Card collapses and is marked done. Player suggests the next unfinished exercise.
Finished	All exercises done, or the user taps “Finish”. Session is saved and the summary opens.

6.3 Rest timer and breathing cue

- The breathing cue is a slowly expanding and contracting circle with the text “Breathe in” / “Breathe out”, on a 4-second in, 6-second out cycle.
- No rest timer after the final set of the final exercise.
- **Timer correctness:** store the rest *end timestamp*, not a remaining-seconds counter. Recompute the remaining time from the clock whenever the app resumes. This keeps the timer right after the screen locks or the app is backgrounded.
- **Locked-screen alert:** when rest starts, schedule a local notification for the end timestamp. Cancel it if the user skips or the app is in the foreground at the end (play sound and vibrate instead).
- **Crash safety:** write each logged set to the database immediately. On launch, if an unfinished session exists, offer to resume it.

7 Reminders

- User picks days of the week and one time. The app schedules repeating local notifications.
- Notification text: “Time to train” plus the name of the least recently performed workout.
- Tapping the notification opens the Library.
- Ask for notification permission the first time the user enables reminders or starts a rest timer, not on first launch.

8 Technical design

8.1 Suggested stack

Concern	Choice
Framework	Flutter (stable channel), Dart with sound null safety.
State management	<code>flutter_riverpod</code> .
Navigation	<code>go_router</code> .
Local database	<code>drift</code> (SQLite). Typed queries and migrations.
Notifications	<code>flutter_local_notifications</code> with <code>timezone</code> .
File import	<code>file_picker</code> . Clipboard via Flutter's built-in <code>Clipboard</code> .
Screen awake	<code>wakelock_plus</code> .
Tests	<code>flutter_test</code> for unit and widget tests.

Check pub.dev for the current version and platform setup steps of each package before adding it.

8.2 Project structure

```
lib/  
  main.dart  
  app/           router, theme  
  core/          shared widgets, utilities  
  features/  
    import/      parser, validator, import + preview screens  
    library/     workout list and detail  
    player/      session controller, player and rest screens  
    history/     session list and detail  
    settings/    settings and reminders  
  data/  
    db/          drift tables, DAOs, migrations  
    models/      plain Dart domain models  
    repositories/ WorkoutRepository, SessionRepository  
    services/    NotificationService, ClockService  
  test/  
    import/      validator tests with JSON fixtures  
    player/      session controller tests
```

8.3 Design rules

- **The validator is pure Dart.** Input: a string. Output: a list of workouts or a list of errors. No Flutter imports, so it is trivial to test.
- **The session controller is pure Dart.** It owns the player state machine and knows nothing about widgets. This is the piece a watch app will drive later.
- **Inject the clock.** The controller takes a `ClockService` so tests can move time forward.
- **UI reads state, sends intents.** Widgets call methods such as `completeSet()` and `skipRest()`; they do not contain workout logic.

8.4 Data model

Table	Columns
workouts	id, name, description, tags, default_rest_seconds, created_at
exercises	id, workout_id, position, name, type, sets, reps, reps_max, duration_seconds, weight, unit, rest_seconds, notes, group_key
sessions	id, workout_id (nullable), workout_name, started_at, finished_at (null while in progress)
set_logs	id, session_id, exercise_name, exercise_position, set_number, reps, weight, unit, duration_seconds, completed_at

Sessions copy the workout and exercise names so that history survives deleting or replacing a workout. “Last time” is looked up by exercise name across all sessions.

8.5 Platform notes

- **Android 13+:** notifications need the runtime `POST_NOTIFICATIONS` permission.
- **Android 12+:** a rest alert that fires on the exact second needs exact-alarm permission. Test this on a real device with battery saver on.
- **iOS:** request notification authorisation; scheduled local notifications fire while the app is suspended.
- Test the locked-screen rest alert on a physical phone early. Emulators hide timing problems.

9 Milestones

#	Milestone	Exit condition
M0	Project setup	App runs on your phone with routing, theme, and empty screens.
M1	Schema and import	Pasting the example JSON saves a workout; bad JSON shows clear errors; validator tests pass.
M2	Library	Saved workouts are listed, viewable, deletable, and exportable.
M3	Player and rest timer	A full workout can be completed; rest alert fires with the screen locked.
M4	Logging and history	Sets are stored; “last time” appears; sessions resume after a crash.
M5	Reminders and settings	Weekly reminders fire on the chosen days.
M6	Personal use	Two weeks of real workouts; fix what annoys you.

9.1 Checklist for today

- ☐ Run `flutter create` and get the app onto your phone.
- ☐ Add `flutter_riverpod` and `go_router`; create the empty screens and routes.
- ☐ Write the domain models: `Workout`, `Exercise`.
- ☐ Write the validator as a pure Dart function with an error type that carries a path and message.
- ☐ Save the example from Section 4 as a test fixture, plus three broken variants.

- ☐ Write validator tests until all four fixtures behave as specified.
- ☐ Build the Import screen: paste, validate, show errors or preview (in memory only for now).

10 Acceptance criteria

1. The example JSON in Section 4 imports without errors and previews three exercises.
2. JSON with a missing required field produces an error naming the workout, exercise, and field.
3. JSON with two separate mistakes reports both in one validation pass.
4. A file with `schemaVersion 2` is rejected with an “update the app” message.
5. Exercises can be performed in any order by tapping them.
6. Completing a set starts a rest timer with the correct duration (exercise value, else workout default, else 90 s).
7. Locking the phone during rest and unlocking it later shows the correct remaining time.
8. A notification with sound or vibration arrives within about two seconds of rest ending while the phone is locked.
9. Killing the app mid-workout and reopening it offers to resume with all logged sets intact.
10. The second time an exercise is performed, the previous reps and weight are shown.
11. Exporting a workout and re-importing it produces an identical workout.
12. A reminder set for a given weekday and time fires on that day.

11 Risks

Risk	Mitigation
Rest alert is late or missing on some Android phones because of battery optimisation.	Prototype in M3 on a real device. Use exact alarms. Document a fallback of keeping the screen on.
Schema needs breaking changes after real use.	<code>schemaVersion</code> from day one; keep an importer per version.
Scope creep before the core is solid.	Anything not in Section 2.1 goes into the backlog below.
Player logic becomes tangled with widgets, blocking the watch app later.	Keep the session controller pure Dart with its own tests.

12 Post-MVP backlog

Roughly in priority order:

1. Supersets and circuits using the reserved `group` field.
2. A copyable AI prompt that produces valid workout JSON, and import from a link or QR code.

3. Simple in-app editor for small changes (rename, change sets or rest).
4. Progress charts per exercise and personal records.
5. Opt-in motivational notifications, ideally based on the user's own streaks.
6. Backup and restore to a file, then optional cloud sync.
7. Wear OS app (covers current Samsung watches), then a native Apple Watch app: rest timer, heart rate, start and finish sets from the wrist.
8. Store release: onboarding, sample workouts, privacy policy, store listings.